Review notes · for you

Operations research: unit check

Optimization · Level 4 · Unit check

Time	Materials
20 min	Paper and a pen. No device needed.

Learning goals

- Linear programming and the simplex method
- Integer programs and branch and bound
- Constraint programming: propagation and search
- Heuristics and trade-offs

Activity: Pick the tool

1. Sort (6 min): On cards: a canteen menu, buses for a trip, an exam timetable, a 300-stop delivery round, a farm plan, a hospital roster, a sudoku, choosing warehouse sites, a study plan, a sports league. Put each under LP, MIP, CP-SAT or heuristic, with one reason.
2. Model (8 min): Pick one card and write its variables (with units), constraints and objective in one paragraph.
3. Report (6 min): List what a decision-maker should see besides the plan: binding limits and shadow prices, the gap, the assumptions, and the trade-offs and fairness limits.

Many problems fit more than one tool; the reason matters more than the label.

Discussion

- When is a relaxation's answer useful, even though it can't be used as it is?
- Why is INFEASIBLE good news for a modeller?
- What does a responsible modeller owe the people affected by a plan?

Check yourself

- Name one problem from your own life, the tool you'd use, and the one number you'd report besides the answer.

Try it this week

Take one decision you care about (a society budget, a timetable, a delivery round) and decide which tool fits it: LP, MIP, CP-SAT or a heuristic. Write the model's variables, constraints and objective in one paragraph, and say what you would report besides the answer: binding limits, the gap, the trade-offs.

Quiz sheet

Operations research: unit check

Optimization · Level 4 · Unit check

Name _____ Class _____ Date _____

1 **Why does a linear program with an optimum always have one at a corner of the feasible region?**

- ◯ Solvers only look at corners
- ◯ The constraints are all equalities
- ◯ Corners always have whole-number values
- ◯ The region is convex and the objective is linear

2 **For a minimisation MIP, the LP relaxation gives an upper bound on the optimal cost.**

- ◯ True
- ◯ False

3 **Match each method to the problem it suits best.**

Simplex (GLOP)	• • • •	Exam slots with all-different rules
Branch and bound (SCIP)	• • • •	The diet problem with fractional portions
CP-SAT	• • • •	A 300-stop delivery route by morning
2-opt local search	• • • •	Whole buses with a booking fee

4 **What does this program print?**

```
found, bound = 1400, 1173.33
gap = (found - bound) / found
print(round(100 * gap))
```

- ◯ 19
- ◯ 84
- ◯ 16

5 **Which statements about shadow prices are true? Choose all that apply.**

- ☐ A binding constraint can have a non-zero shadow price
- ☐ A shadow price holds for any change, however large
- ☐ Shadow prices tell you which variables must be whole numbers
- ☐ A constraint with slack has a shadow price of zero
- ☐ In the garden, 8 × 25 + 18 × 5 + 14 × 0 equals the best profit, 290

6 **Complete the sentence.**

Branch and bound splits on a _____ variable and _____ every branch whose relaxation can't beat the best plan found.

Word bank: prunes · doubles · fractional · continuous

7 **Put the steps of branch and bound in order.**

- [ ] Prune branches that can’t beat the best plan found
- [ ] Solve the LP relaxation
- [ ] Pick a fractional variable and branch on it
- [ ] Stop when no branch can do better: the best plan is proved
- [ ] Solve each branch’s relaxation

8 **This exam-slot model should use as few slots as possible, but it always reports 1. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
1  slot = {e: model.new_int_var(1, 8, e) for e in exams}
2  for taken in pupils:
3      model.add_all_different([slot[e] for e in taken])
4  last = model.new_int_var(1, 8, "last")
5  for e in exams:
6      model.add(slot[e] >= last)
7  model.minimize(last)
```

---

9 **Simulated annealing started at a very low temperature behaves much like hill climbing.**

◯ True ◯ False

10 **Complete the 2-opt move: reverse the stretch from i to j, and keep it if the route is shorter.**

```
new = r[:i] + r[i:j + 1][ _____ ] + r[ _____ :]
if length(new) _____ length(r):
    r = new
```

Word bank: `i` `>` `::-1` `j + 1` `::1` `<`

Check your answers

Operations research: unit check

Optimization · Level 4 · Unit check

1 **The region is convex and the objective is linear**

A linear objective is a family of parallel lines; pushed as far as it can go over a convex region, it last touches a vertex (or a whole edge containing vertices).

2 **False**

It gives a lower bound: the relaxation allows every integer plan and more, so its minimum can only be lower or equal (1,173.33 against 1,400 for the trip).

3 **Simplex (GLOP) — The diet problem with fractional portions**
Branch and bound (SCIP) — Whole buses with a booking fee
CP-SAT — Exam slots with all-different rules
2-opt local search — A 300-stop delivery route by morning

Continuous linear models suit simplex; whole numbers and fixed costs suit MIP; logical and combinatorial rules suit CP-SAT; huge routing instances under a time limit suit local search, with a gap to show how good the answer is.

4 **16**

(1,400 − 1,173.33) ÷ 1,400 ≈ 0.162, so it prints 16: the trip's best plan was, at first sight, within 16% of the relaxation's bound.

5 **A binding constraint can have a non-zero shadow price; A constraint with slack has a shadow price of zero; In the garden, 8 × 25 + 18 × 5 + 14 × 0 equals the best profit, 290**

Binding constraints carry the prices; slack ones are free (complementary slackness), and the prices value the limits at exactly the optimum (strong duality: 200 + 90 + 0 = 290).

6 **Branch and bound splits on a fractional variable and prunes every branch whose relaxation can't beat the best plan found.**

A fractional value (2.13 buses) is where the relaxation breaks the integer rule, so that's where to split; pruning branches that can't win is what keeps the tree small.

7 **Top to bottom: 4, 1, 2, 5, 3**

The relaxation gives the first bound; branching removes a fraction; each branch is bounded by its own relaxation; pruning compares bounds with the incumbent; when nothing is left open, the incumbent is optimal.

8 **Line 6: `model.add(slot[e] <= last)`**

last must be at least every exam's slot, so that it equals the last slot used.

9 **True**

At low T, e^(−d/T) is almost zero for any worse move, so only improving moves are kept: hill climbing.

10

```
new = r[:i] + r[i:j + 1][::-1] + r[j + 1:]
if length(new) < length(r):
    r = new
```

[::-1] reverses the slice, r[j + 1:] is the rest of the route after it, and < keeps only shorter routes.